Dataproc Pipeline Testing, Monitoring & Best Practices

A Practical Handbook for Building Reliable Spark Pipelines on Google Cloud

Covers: Unit testing PySpark transforms with PyTest • Cloud Monitoring metrics & log-based alerts • Persistent History Server • Production reliability patterns • A full hands-on demo you can run yourself (PyTest, gcloud CLI, and a companion notebook)

Prepared for GCP Data Engineering Training

Reflects Google Cloud naming as of July 2026 — Managed Service for Apache Spark

Table of Contents

1. Why Pipeline Engineering Discipline Matters

Getting a PySpark job to run once on Dataproc is the easy part. The real engineering work — the part that separates a demo script from a production pipeline — is making sure it keeps working correctly as the code changes, the data changes, and the job runs unattended at 2 AM every night for the next two years. That's what this handbook is about: three practices that, together, are what 'production-ready' actually means for a Dataproc pipeline.

- **Testing** — catching a broken transformation before it ships, not after it silently corrupts a downstream table.
- **Monitoring** — knowing a job failed, slowed down, or is about to run out of memory, without waiting for someone to notice bad numbers in a dashboard three days later.
- **Best practices** — the accumulated, often hard-won lessons about how to structure jobs, dependencies, and cluster/batch configuration so pipelines are reproducible and debuggable rather than fragile and mysterious.

None of this requires exotic tooling. Testing uses the same PyTest most Python developers already know. Monitoring uses Cloud Monitoring and Cloud Logging, already running underneath every Dataproc job whether you look at them or not. The gap is almost always *habit*, not *tooling* — this handbook is designed to make the habit concrete and repeatable.

2. Core Concepts & Glossary

Term	Plain-English Definition
Unit test	A test that checks one small piece of logic in isolation — e.g. does this one transform function correctly compute a derived column? — without spinning up a cluster or touching real data.
PyTest	The standard Python testing framework, used here to unit-test PySpark transformation logic. Tests run locally using a small, local SparkSession — no Dataproc cluster or batch required.
Fixture (PyTest concept)	A reusable setup function (e.g. one that creates a local SparkSession, or a small sample DataFrame) that multiple tests can share, declared once and injected into each test function.
Local SparkSession	A SparkSession running in local mode (master="local[*]") entirely inside the test process, with no cluster or cloud resources involved — what makes unit-testing Spark code fast and free.
Integration test	A test that exercises the real pipeline end-to-end against real (or realistic, small-scale) infrastructure — e.g. actually submitting a small Dataproc Serverless batch against a scratch BigQuery dataset and checking the output.
Data quality check / assertion	A runtime check embedded in a pipeline (row count within expected range, no unexpected nulls in a required column, referential integrity against another table) that fails the job loudly rather than silently producing bad data downstream.
Idempotency	A pipeline design property: running the same job twice with the same input produces the same result, rather than duplicating rows or compounding an error. Critical for any job that might be retried after a failure.
Persistent History Server (PHS)	A long-lived Dataproc component that retains Spark event logs (task timings, DAG execution, stage details) even after the ephemeral cluster or Serverless batch that generated them is gone — essential because Dataproc Serverless and ephemeral clusters otherwise lose this data the moment they're torn down.
Component Gateway	The Dataproc feature that exposes web UIs for cluster components (YARN Resource Manager, Spark History Server, Jupyter) securely through the console, without needing to open firewall ports or SSH tunnel manually.
Spark UI / Spark History Server UI	The web interface showing a Spark job's execution DAG, stage/task timings, shuffle data volumes, and executor resource usage — the primary tool for diagnosing 'why is this job slow' questions.
Cloud Monitoring metric	A time-series measurement (e.g. YARN pending memory, job duration, batch state) automatically collected for Dataproc clusters, jobs, and Serverless batches, viewable in Metrics Explorer or custom dashboards.

Term	Plain-English Definition
Custom metric source	An optional, explicitly-enabled category of deeper metrics (spark, yarn, hdfs, spark-history-server, etc.) beyond the default set, enabled via --metric-sources on cluster creation.
Log-based metric / alerting policy	A custom Cloud Monitoring metric derived from matching Cloud Logging entries (e.g. count of FAILED job states), used as the basis for an alerting policy that notifies a team when a threshold is crossed.
Image version pinning	Explicitly specifying a Dataproc image version (e.g. --image-version=2.2) rather than accepting whatever the current default is, so a production job's environment doesn't silently change underneath it when Google ships a new default image.
Uber jar	A single JAR bundling a job's own code together with any non-cluster-provided dependencies, so job submission doesn't depend on manually distributing separate dependency JARs to every node.
Workflow Template	A Dataproc feature that defines a reusable sequence of cluster-creation + job-submission + cluster-deletion steps, so an ephemeral 'spin up, run, tear down' pattern is fully automated and repeatable rather than scripted ad hoc.
Data skew	When data isn't evenly distributed across partitions, causing some Spark tasks to take far longer than others — one of the most common real-world causes of a 'mysteriously slow' job, visible in the Spark UI as a few very long-running tasks.
Dynamic allocation	Spark's ability to add/remove executors during a job's execution based on workload, rather than requesting a fixed number up front — generally recommended over manually lifting-and-shifting a fixed spark.executor.instances setting from an on-prem cluster.

3. Testing Strategy for Spark Pipelines

The single highest-leverage habit: **separate your transformation logic from your job's I/O and orchestration**, so the logic can be unit-tested with a local SparkSession in seconds, with no cluster, no cloud credentials, and no network access required.

Structure your job for testability

```
# transforms.py -- pure functions, no I/O, easy to test
from pyspark.sql import DataFrame
from pyspark.sql.functions import col, upper

def filter_engineering(df: DataFrame) -> DataFrame:
    return df.filter(col("department") == "Engineering")

def add_department_upper(df: DataFrame) -> DataFrame:
    return df.withColumn("department_upper", upper(col("department")))


# job.py -- thin orchestration layer that reads, calls transforms.py, writes
from pyspark.sql import SparkSession
from transforms import filter_engineering, add_department_upper

def main():
    spark = SparkSession.builder.appName("EmployeesETL").getOrCreate()
    df = spark.read.format("bigquery").option("table", "PROJECT_ID.hr_demo.employees"
).load()
    result = add_department_upper(filter_engineering(df))
    result.write.format("bigquery").option("table", "PROJECT_ID.hr_demo.output").save
()

if __name__ == "__main__":
    main()
```

Nothing in `transforms.py` *touches BigQuery, Cloud Storage, or a real cluster — which is exactly what makes it fast and cheap to test.*

Unit test with PyTest and a local SparkSession

```
# test_transforms.py
import pytest
from pyspark.sql import SparkSession
from transforms import filter_engineering, add_department_upper

@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder.master("local[*]").appName("tests").getOrCreate()

def test_filter_engineering(spark):
    data = [(1, "Riya", "Engineering"), (2, "Arjun", "Sales")]
    df = spark.createDataFrame(data, ["employee_id", "full_name", "department"])
    result = filter_engineering(df)
    assert result.count() == 1
    assert result.collect()[0]["full_name"] == "Riya"

def test_add_department_upper(spark):
    data = [(1, "Engineering")]
    df = spark.createDataFrame(data, ["employee_id", "department"])
    result = add_department_upper(df)
    assert result.collect()[0]["department_upper"] == "ENGINEERING"
```

```
pytest test_transforms.py -v
```

Add data quality assertions inside the pipeline itself

Unit tests catch logic bugs before deployment; data quality assertions catch bad *data* at runtime, which unit tests structurally cannot do.

```
def validate_output(df: DataFrame, min_rows: int = 1) -> DataFrame:
    row_count = df.count()
    if row_count < min_rows:
        raise ValueError(f"Expected at least {min_rows} rows, got {row_count}")
    null_emails = df.filter(col("email").isNull()).count()
    if null_emails > 0:
        raise ValueError(f"Found {null_emails} rows with null email -- failing job")
    return df
```

Integration testing (the layer unit tests can't cover)

Unit tests never touch the BigQuery connector, IAM, or actual Dataproc Serverless behavior — that needs a real, small-scale end-to-end run. Keep a scratch dataset/table specifically for this, submit the real job against a tiny fixture input, and assert on the real output table. Section 7 walks through this as part of the hands-on demo.

4. Monitoring & Observability

Dataproc gives you three complementary layers of visibility, each answering a different question.

Layer	Question it answers	Where to look
Cloud Monitoring metrics	“Is this job/cluster healthy right now, and how does it compare to last week?”	Metrics Explorer, custom dashboards, alerting policies on Dataproc Cluster / Job / Batch resources.
Cloud Logging	“What exactly happened, in what order, and what was the error message?”	Logs Explorer, enabled by default on clusters and Serverless batches; also the source for log-based metrics.
Spark UI / Persistent History Server	“Why is this specific job slow, and which stage/task is the bottleneck?”	Component Gateway (live jobs) or a PHS cluster (for jobs whose original cluster/batch no longer exists).

Set up the Persistent History Server (essential for ephemeral/serverless jobs)

Without this, a Dataproc Serverless batch's Spark event logs disappear the moment the batch's ephemeral compute is torn down — exactly when you'd want to look back at a failed job.

```
# 1. Point the batch at a shared GCS location to WRITE its event logs
gcloud dataproc batches submit pyspark job.py \
  --region=${REGION} \
  --properties=spark.eventLog.enabled=true,\
spark.eventLog.dir=gs://${BUCKET}/spark-event-logs/

# 2. Create a small, separate PHS cluster to READ those logs after the fact --
# spark.history.fs.logDirectory is a CLUSTER setting; it is not a valid batch
# property, so it must only appear here, not in the batch submit command above.
gcloud dataproc clusters create history-server \
  --region=${REGION} --single-node --enable-component-gateway \
  --properties=spark:spark.history.fs.logDirectory=gs://${BUCKET}/spark-event-logs/
```

Open the PHS's Spark History Server UI (via Component Gateway in the console) any time afterward to inspect completed jobs' DAGs, stage timings, and shuffle volumes — even though the batch that ran them is long gone.

Enable richer Cloud Monitoring metrics

```
gcloud dataproc clusters create my-cluster \
  --region=${REGION} \
  --metric-sources=spark,yarn,spark-history-server
```

Dataproc Serverless batches collect core Spark metrics by default; cluster-based Dataproc requires opting in per metric source as shown above. All are viewable under the Cloud Dataproc Cluster / Job / Batch resource types in Metrics Explorer.

Build a log-based alert for job failures

```
gcloud logging metrics create dataproc_job_failures \
  --description="Counts failed Dataproc jobs/batches" \
  --log-filter='resource.type=("cloud_dataproc_cluster" OR "cloud_dataproc_batch") '\
'AND severity=ERROR AND jsonPayload.message:"FAILED"'
```

Then: Monitoring → Alerting → Create Policy → select the `logging/user/dataproc_job_failures` metric → threshold ≥ 1 → add a notification channel.

> **NOTE:** Google Cloud also offers a pre-GA **Gemini Cloud Assist Investigations** feature that analyzes failed or slow Dataproc jobs and suggests root causes automatically — worth trying as a first-pass triage tool, though treat its suggestions as a starting hypothesis, not a verdict, especially while the feature is pre-GA.

5. Production Reliability Patterns

Pin your image version

```
gcloud dataproc clusters create my-cluster \
  --region=${REGION} \
  --image-version=2.2   # a specific major.minor, not "whatever's default today"
```

Without this, a new default image can silently change your job's Spark/library versions between runs — test new minor versions deliberately via a preview image before adopting them in production.

Bundle dependencies with an uber jar (or a pinned Python environment)

For PySpark, the equivalent discipline is pinning a requirements file and, for anything beyond pure PyPI packages, building a custom container image or Conda environment archive rather than installing packages ad hoc on cluster startup.

Design writes to be idempotent

```
# Prefer overwrite-by-partition or MERGE semantics over blind append,
# so a retried job doesn't duplicate data:
result.write.format("bigquery") \
    .option("table", "PROJECT_ID.hr_demo.daily_summary") \
    .option("writeMethod", "direct") \
    .mode("overwrite") \
    .save()
```

Automate the ephemeral cluster lifecycle with Workflow Templates

```
gcloud dataproc workflow-templates create daily-etl --region=${REGION}

gcloud dataproc workflow-templates set-managed-cluster daily-etl \
  --region=${REGION} --cluster-name=daily-etl-cluster \
  --image-version=2.2 --num-workers=2

gcloud dataproc workflow-templates add-job pyspark job.py \
  --step-id=run-etl --workflow-template=daily-etl --region=${REGION}

# Triggering this template creates the cluster, runs the job, then tears the cluster
# down automatically -- no separate cleanup step to forget.
gcloud dataproc workflow-templates instantiate daily-etl --region=${REGION}
```

Watch for data skew proactively

In the Spark UI, a stage where a handful of tasks take dramatically longer than the rest is the signature of data skew — usually a join or group-by key with a few disproportionately common values. Common fixes: salting the skewed key, broadcasting a small side of a join, or repartitioning explicitly before the skewed operation.

6. IAM Roles You Need to Know

Role (display name)	Role ID	What it lets you do
Dataproc Editor	roles/dataproc.editor	Submit jobs/batches, create Workflow Templates, manage clusters.
Dataproc Viewer	roles/dataproc.viewer	Read-only access to job status, logs, and cluster metadata — a good fit for a monitoring/on-call role that shouldn't be able to submit or modify jobs.
Monitoring Editor	roles/monitoring.editor	Create dashboards, alerting policies, and log-based metrics.
Logs Configuration Writer	roles/logging.configWriter	Create log-based metrics used as the basis for failure alerts.
Storage Object Admin (on the event-log bucket)	roles/storage.objectAdmin	Needed by the job's service account to write Spark event logs to the shared GCS location used by the Persistent History Server.

7. Hands-On Demo — Step by Step

This demo builds a small transform module with unit tests, runs them locally, then submits the real job as a Dataproc Serverless batch configured for Persistent History Server logging, and sets up a failure alert.

> **NOTE:** Time required: ~30–40 minutes.

Prerequisites

- Python 3 with `pip install pyspark pytest` locally (or in this notebook's environment) for the unit-testing half — no cloud resources needed for this part.
- A Google Cloud project (e.g. `himanshugcpproject`) with Dataproc Editor and Monitoring Editor on your account.
- A Cloud Storage bucket for Spark event logs.

STEP 1 — Write the transform module and its unit tests

Create `transforms.py` and `test_transforms.py` exactly as shown in Section 3, then run:

```
pytest test_transforms.py -v
```

You should see both tests pass in well under a second — no cluster, no cloud calls, no cost.

STEP 2 — Create the event-log bucket and submit the real job

```
export PROJECT_ID="himanshugcpproject"
export REGION="us-central1"
export BUCKET="${PROJECT_ID}-pipeline-demo"

gcloud storage buckets create gs://${BUCKET} --location=${REGION}

gcloud dataproc batches submit pyspark job.py \
  --project=${PROJECT_ID} --region=${REGION} \
  --deps-bucket=${BUCKET} --version=2.2 \
  --py-files=transforms.py \
  --properties=spark.eventLog.enabled=true,\
spark.eventLog.dir=gs://${BUCKET}/spark-event-logs/
```

STEP 3 — Stand up a Persistent History Server and inspect the run

```
gcloud dataproc clusters create history-server \
  --project=${PROJECT_ID} --region=${REGION} \
  --single-node --enable-component-gateway \
  --properties=spark:spark.history.fs.logDirectory=gs://${BUCKET}/spark-event-logs/
```

Console → Dataproc → Clusters → history-server → Web Interfaces → Spark History Server. Open your batch's run and look at the Stages tab — this is where you'd spot data skew or an unexpectedly long shuffle in a real investigation.

STEP 4 — Create a failure alert

```
gcloud logging metrics create dataproc_job_failures \
  --description="Counts failed Dataproc jobs/batches" \
  --log-filter='resource.type=("cloud_dataproc_cluster" OR "cloud_dataproc_batch") '\
'AND severity=ERROR'
```

Console → Monitoring → Alerting → Create Policy, using the `logging/user/dataproc_job_failures` metric with a threshold of ≥ 1 over 5 minutes, and add an email notification channel.

STEP 5 — Verify the alert fires (intentionally break something)

Submit a batch referencing a table that doesn't exist, to trigger a real failure and confirm the alert notifies you within a few minutes:

```
gcloud dataproc batches submit pyspark job.py \
  --project=${PROJECT_ID} --region=${REGION} \
  --deps-bucket=${BUCKET} --version=2.2 \
  --properties=spark.sql.shuffle.partitions=1 \
  -- --table=hr_demo.nonexistent_table
```

8. Troubleshooting Common Issues

Symptom	Likely Cause / Fix
Unit tests hang or fail to start a local SparkSession	Usually a Java version mismatch — PySpark needs a compatible JDK installed locally (check the PySpark version's supported Java range) even though no cluster is involved.
Spark History Server UI shows no applications	Confirm the PHS cluster's spark.history.fs.logDirectory exactly matches the spark.eventLog.dir the job was submitted with — a path mismatch is the most common cause.
Log-based metric never increments despite real failures	Double-check the --log-filter resource.type and severity match what your job's failures actually log as — filters are exact-match; test with a broader filter first, then narrow it.
Unit tests pass but production job still produces wrong data	Unit tests only cover the logic paths you wrote tests for. Add a data quality assertion (Section 3) as a second, runtime layer of defense unit tests structurally can't provide.
Job intermittently fails only under production data volume	Classic data skew signature. Check the Spark UI's Stages tab for a small number of disproportionately long tasks, and consider salting the join/group-by key or repartitioning.
Workflow Template cluster fails to delete after job completion	Check the template's cluster-deletion step wasn't skipped due to a job failure upstream — by default a failed job step can leave the managed cluster running; review the template's failure policy.

9. Best Practices Checklist

- Separate transformation logic (pure functions) from I/O and orchestration — this single restructuring is what makes fast, cheap unit testing possible at all.
- Add data quality assertions as a second line of defense inside the pipeline itself; unit tests cannot catch bad production data, only bad logic.
- Always configure spark.eventLog.dir to a shared GCS location and stand up (or reuse) a Persistent History Server — without it, ephemeral/Serverless jobs lose all diagnostic history the moment compute is torn down.
- Pin image versions explicitly in production; test new minor versions via preview images before adopting them.
- Design every write to be idempotent (overwrite-by-partition, MERGE) so a safe retry after failure never duplicates or corrupts data.
- Build at least one failure alert (log-based metric + alerting policy) before a pipeline goes to production — don't wait for someone to notice missing data downstream.
- Use Workflow Templates for the ephemeral cluster pattern rather than hand-scripting create/submit/delete — it removes an entire class of 'forgot to delete the cluster' cost incidents.
- Treat the Spark UI / History Server as a routine check after every significant job, not just a debugging tool reached for only when something is visibly broken — catching a slow trend early is cheaper than firefighting a production incident later.

10. Companion Notebook & Cleanup

A companion notebook (`dataproc_testing_monitoring_demo.ipynb`) accompanies this handbook. It writes the transform module and PyTest suite, runs the tests inline and prints pass/fail results, then submits the real batch with Persistent History Server logging configured, and creates the log-based failure metric — covering the full loop from Section 7 in one place.

Cleanup

```
gcloud dataproc clusters delete history-server --region=${REGION} --quiet
gcloud logging metrics delete dataproc_job_failures --quiet
gcloud storage rm --recursive gs://${BUCKET}
gcloud storage buckets delete gs://${BUCKET}
```

11. Quick Reference & Further Reading

One-line summary

```
Separate logic from I/O -> unit-test logic with PyTest + local SparkSession (free, fa
st) ->
add runtime data quality assertions (catches bad data, not just bad logic) ->
log Spark events to GCS + run a Persistent History Server (keeps diagnostics after
ephemeral compute is torn down) -> build a failure alert before going to production.
```

Official documentation to bookmark

- Dataproc best practices for production — docs.cloud.google.com/dataproc/docs/guides/dataproc-best-practices
- Dataproc monitoring and troubleshooting tools — docs.cloud.google.com/dataproc/docs/support/troubleshoot-monitor
- Managed Service for Apache Spark metrics — docs.cloud.google.com/dataproc/docs/guides/dataproc-metrics
- PyTest documentation — docs.pytest.org

End of handbook. Pair this with the companion notebook for the live demo, and the Troubleshooting table in Section 8 during Q&A.;